

B7 Series B

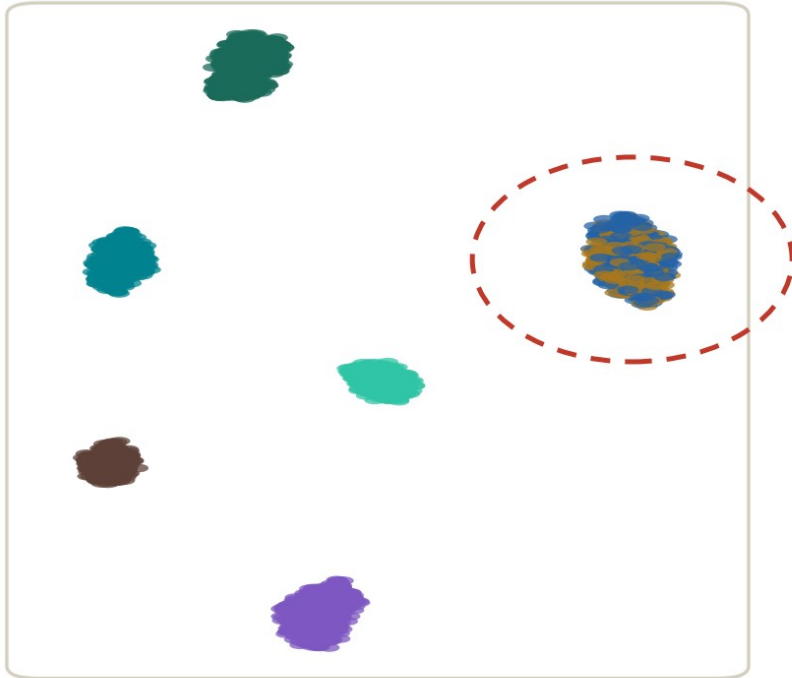
PCA and choosing dimensions: how many PCs, really

Three ways to choose n_{PC} — and why the choice is more forgiving than you fear.

About 32 min | Prerequisites: B6 | Data: PBMC 3k

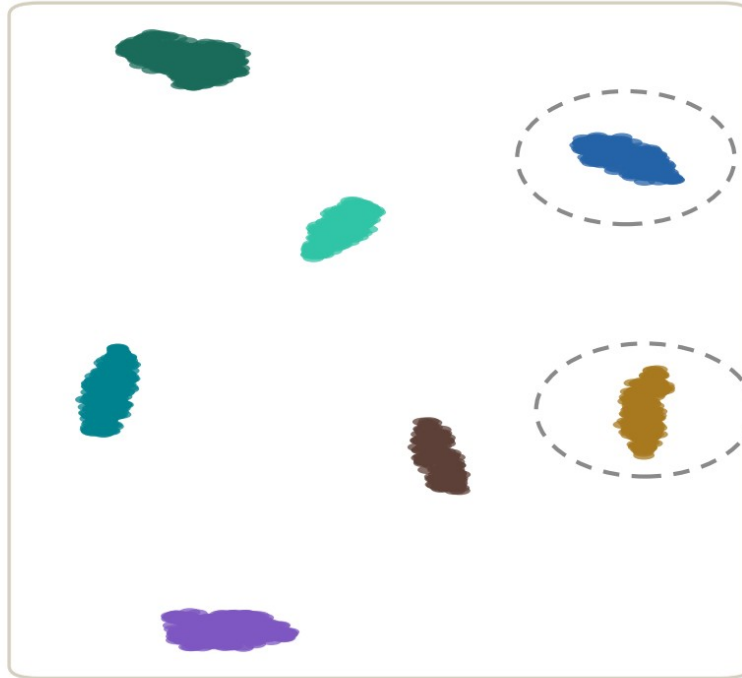
Same data, only nPC changed

nPC = 5



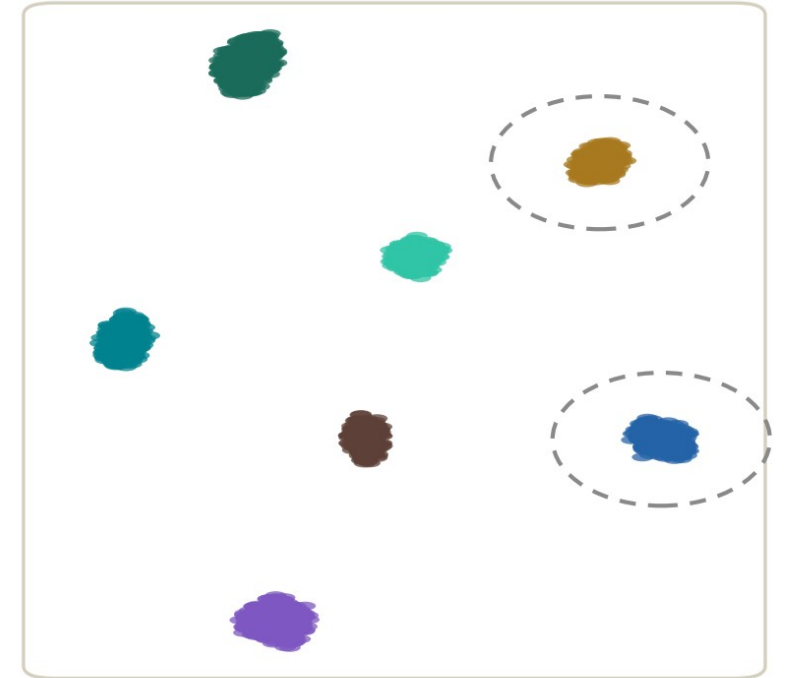
blurred: two subtypes fused

nPC = 10



structure intact

nPC = 30



nearly identical to nPC=10

same data, same pipeline, only nPC changed — why do 10 and 30 look the same?

simulated data

nPC = 5 is blurred; 10 and 30 are nearly identical. Why is taking more usually the safe side?

WHERE THIS EPISODE STARTS

**Too few PCs and information never reaches
downstream;
too many just adds a little noise.**

This asymmetry underlies every judgement in this episode — all three methods come back to it.

What you will learn

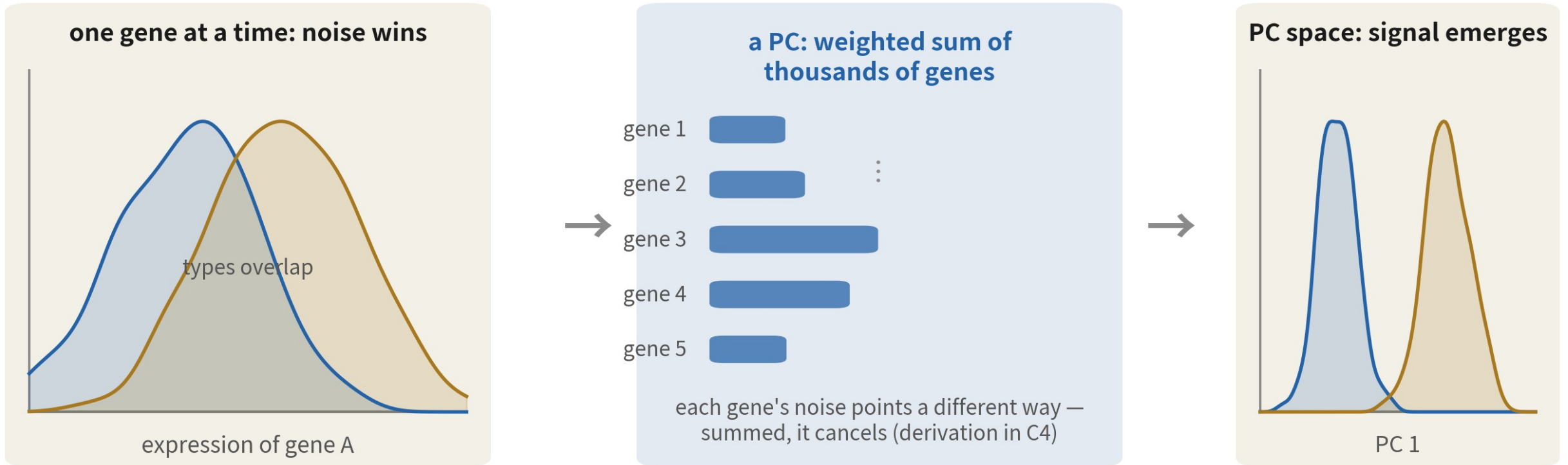
- 1 Run RunPCA and read its three outputs: loadings, scatter, DimHeatmap
- 2 Choose nPC with three methods: ElbowPlot, quantitative cutoffs, stability sweep
- 3 State what too few and too many PCs each cost, and block both traps with one-line checks

01

Why reduce dimensions at all

20,000 axes, mostly noise — intuition before code

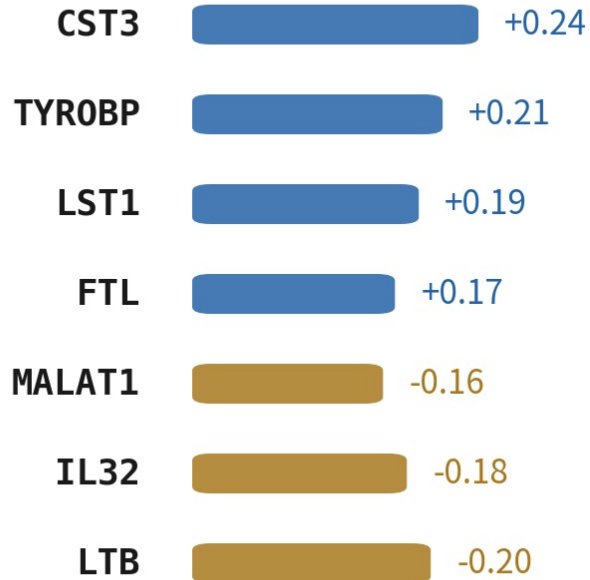
Average the noise out, and signal surfaces



Two types that overlap on one gene separate on a weighted sum of thousands — the noise cancels (derivation in C4)

A PC is a weighted combination of genes

PC1 loadings: one weight per gene



a cell's PC1 score

$$= 0.24 \times \text{CST3} + 0.21 \times \text{TYROBP} + \dots \\ - 0.20 \times \text{LTB} - 0.18 \times \text{IL32} - \dots$$

genes that vary together form one expression program;
same-sign genes switch on in the same cells.

13,714 genes → 50 PCs: new coordinates, same information

One weight (loading) per gene; genes that vary together form one expression program

CORE IDEA

**PCA is a change of coordinates:
from 20,000 genes to 50 expression programs.**

No information is invented — it is re-ranked by variance explained. How many to keep is the actual decision.

02

RunPCA and a tour of its output

One line of code, three ways to read the result

Pick up from B6, run PCA

```
library(Seurat)
set.seed(1234)

pbm c <- readRDS("output/pbm c_b06_norm.rds") # B6 的產物
# 沒跑 B6? 從 B5 起手, 重跑 normalization 三行即可:
# pbm c <- readRDS("output/pbm c_b05_qc.rds") |> NormalizeData() |>
#       FindVariableFeatures() |> ScaleData()

pbm c <- RunPCA(pbm c, features = VariableFeatures(object = pbm c))
```

features = the 2,000 HVGs; npcs defaults to 50 — compute plenty now, choose later

Output 1: each PC's recipe

```
print(pbm c[["pca"]], dim s = 1:5, nfeatures = 5)
```

PC_1

Positive: CST3, TYROBP, LST1, AIF1, FTL

Negative: MALAT1, LTB, IL32, IL7R, CD2

PC_2

Positive: CD79A, MS4A1, TCL1A, HLA-DQA1, HLA-DQB1

Negative: NKG7, PRF1, CST7, GZMB, GZMA

(PC_3 – PC_5 略)

PC1: myeloid markers on one end, lymphoid on the other — the largest divide in the data

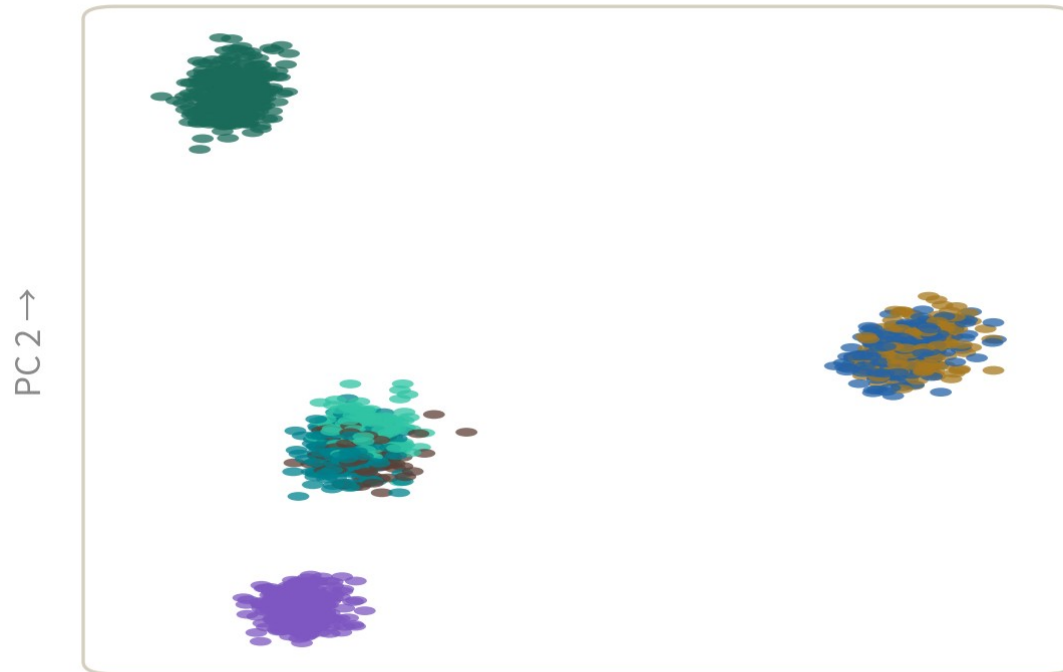
Outputs 2 and 3: draw them

```
# loadings 的圖版: 正負兩端的 top 基因  
VizDim Loadings(pbm c, dims = 1:2, reduction = "pca")  
  
# 細胞在 PC1 x PC2 平面上的位置  
DimPlot(pbm c, reduction = "pca") + NoLegend()
```

The reduction argument picks the space to plot — the same functions serve UMAP later

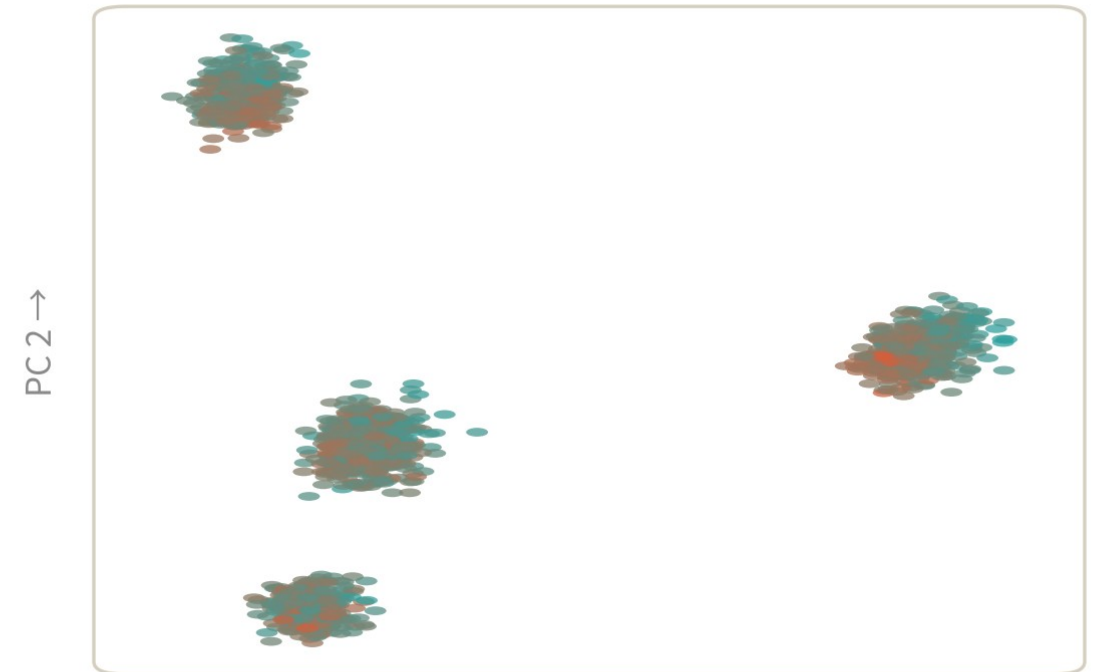
PC1 x PC2: colour by type, then by depth

coloured by cell type



PC1 / PC2 capture the two largest type contrasts

coloured by depth (nCount)



if B6 normalization worked, depth should not own a PC

simulated data

Left: the top PCs capture the biggest type contrasts. Right: depth should mix into clusters, not own an axis

One-line check: is PC1 a depth axis?

```
cor(pbm c$nCount_RNA, Embeddings(pbm c, "pca")[, 1])  
  
# 順手看 PC1 的 top 20 loadings 裡核糖體基因佔多少  
top.pc1 <- names(sort(abs(Loadings(pbm c, "pca")[, 1]),  
                      decreasing = TRUE))[1:20]  
mean(grep("^RP[LS]", top.pc1))
```

```
[1] 0.11
```

```
[1] 0.05
```

Low correlation, low ribosomal share — PC1 is biology. These two lines vaccinate against trap 2

What you have at this point

A pca reduction



50 new coordinates
per cell, stored in the
object's reductions

A stack of recipes



loadings per PC: which
genes, which direction

One number to decide



how many PCs go
downstream — the
rest of this episode

03

DimHeatmap: watch the signal decay

Signal does not vanish — it fades

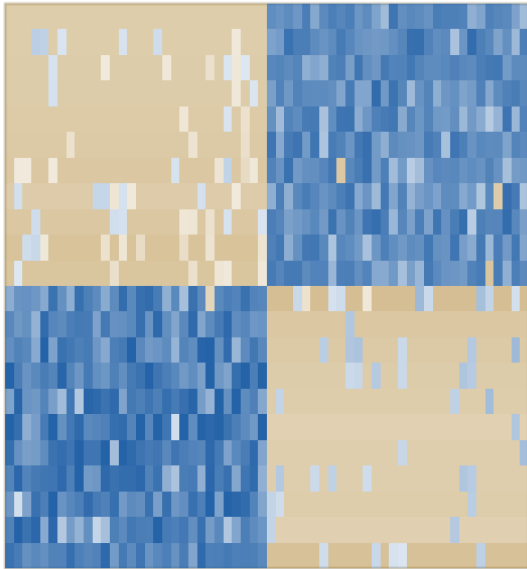
One heatmap of extreme cells per PC

```
# 單看 PC1：分數最極端的 500 顆細胞 x top loading 基因  
DimHeatmap(pbm c, dims = 1, cells = 500, balanced = TRUE)  
  
# 一次看前 15 個，衰減的過程一目瞭然  
DimHeatmap(pbm c, dims = 1:15, cells = 500, balanced = TRUE)
```

balanced = TRUE samples both extremes, so both directions of the program show

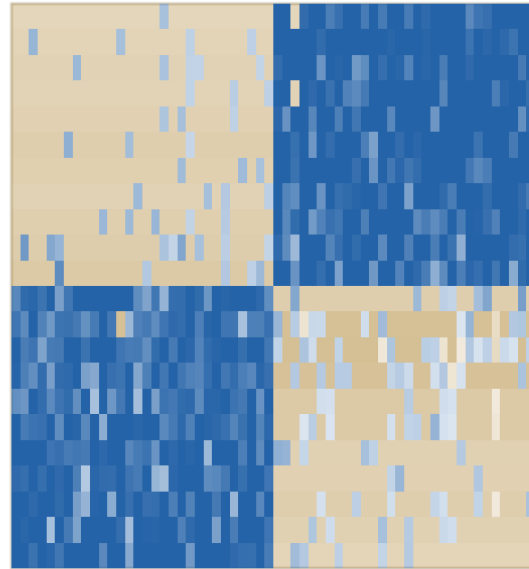
From sharp blocks to salt-and-pepper

PC 1



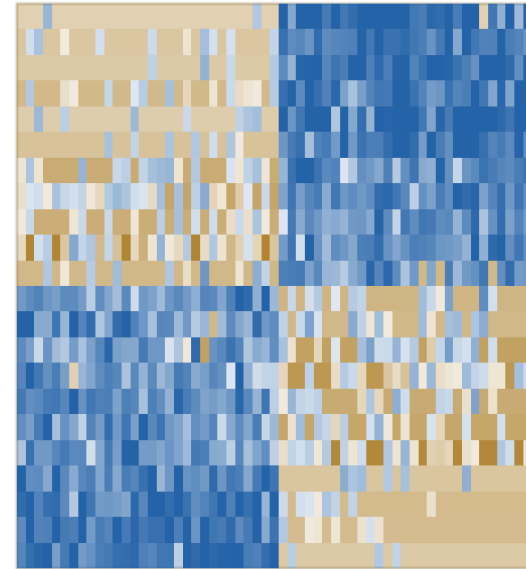
sharp blocks: strong signal

PC 5



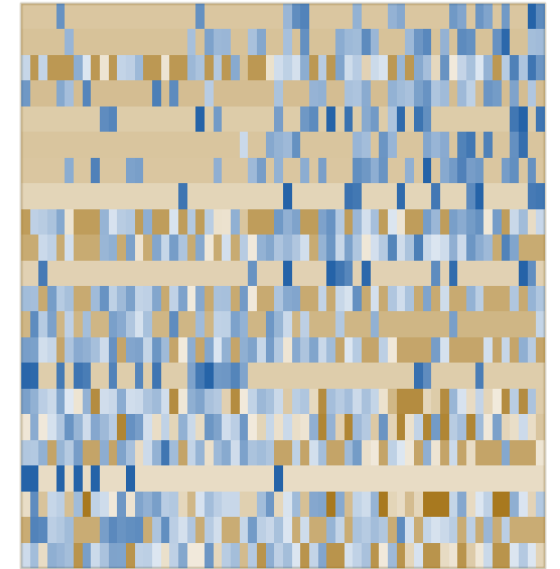
structure remains

PC 7



weak but real —
the subtypes live here

PC 15



salt-and-pepper: noise

rows = top loading genes, columns = extreme cells; signal decays into noise along the PCs

simulated data

Weak is not none — subtype contrasts often live in those mid-range, weak-but-structured PCs

04

ElbowPlot and quantitative cutoffs

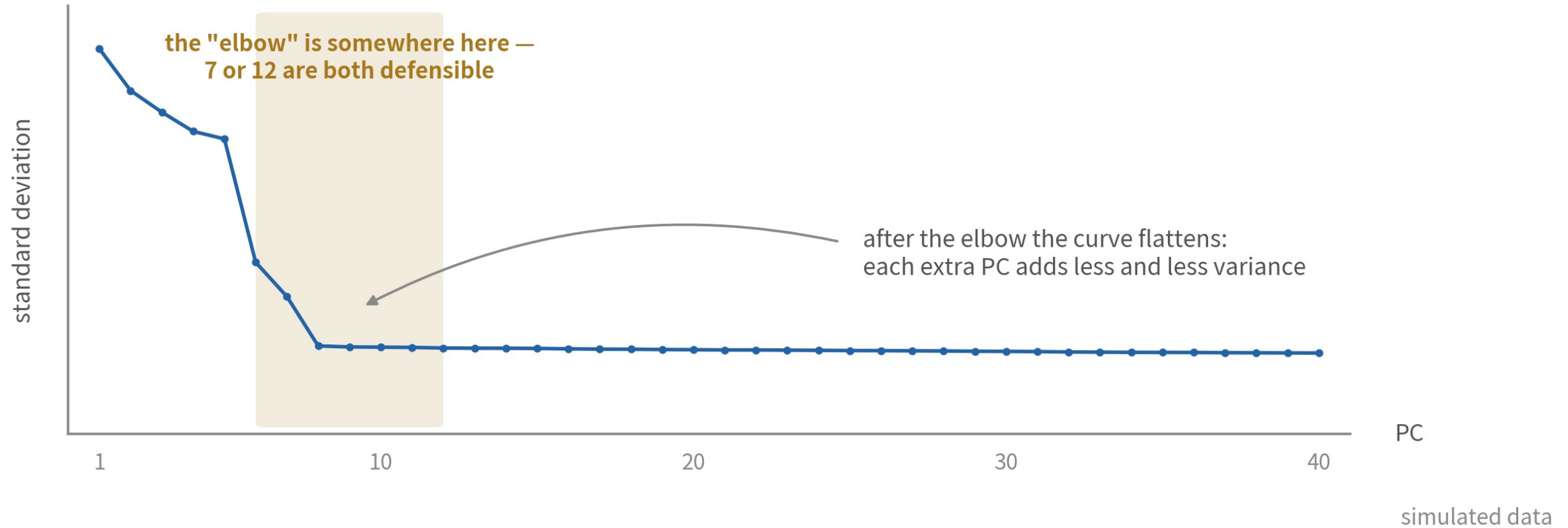
Eyeball the elbow, then compute a number you can publish

Method 1: ElbowPlot

```
ElbowPlot(pbm c, ndims = 50)
```

y is each PC's standard deviation; ndims defaults to 20 — set 50 to see the whole tail

How to read the elbow



The elbow is a range, not a point — 7 and 12 are both defensible answers

Method 2: turn the elbow into numbers

```
pct <- pbm c[["pca"]]@ stdev / sum (pbm c[["pca"]]@ stdev) * 100
cum u <- cum sum (pct)

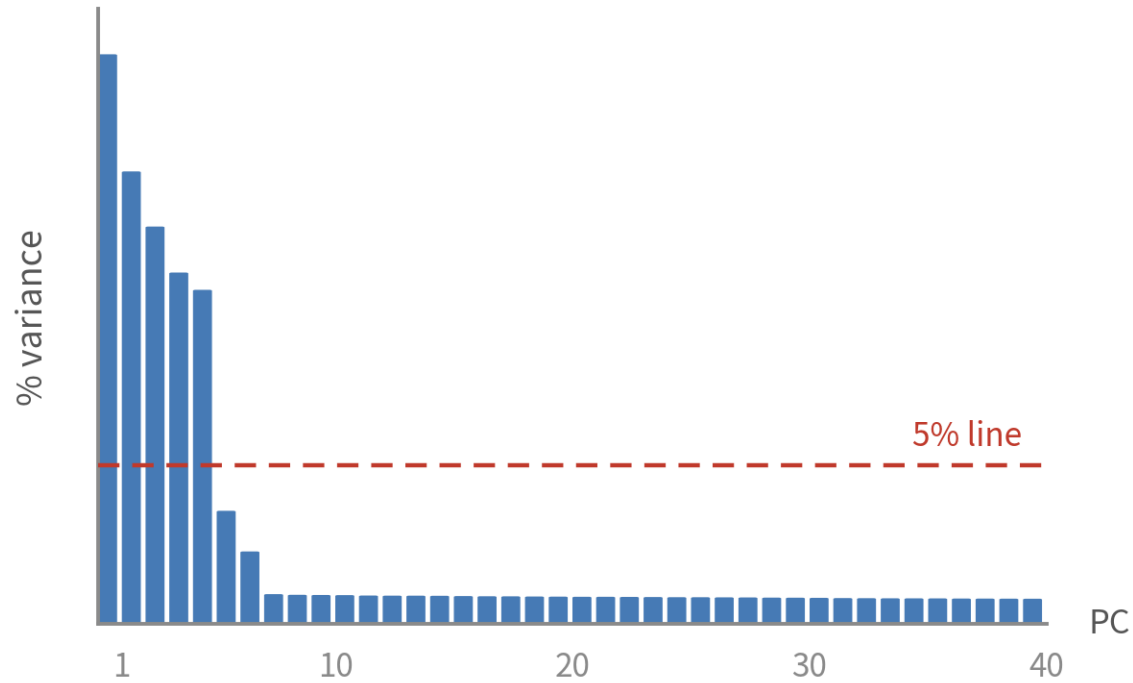
co1 <- which(cum u > 90 & pct < 5)[1] # 累積 90%、單 PC < 5%
co2 <- sort(which(diff(pct) < -0.1), decreasing = TRUE)[1] + 1
c(cumulative90 = co1, drop0.1 = co2)
```

cumulative90	drop0.1
44	13

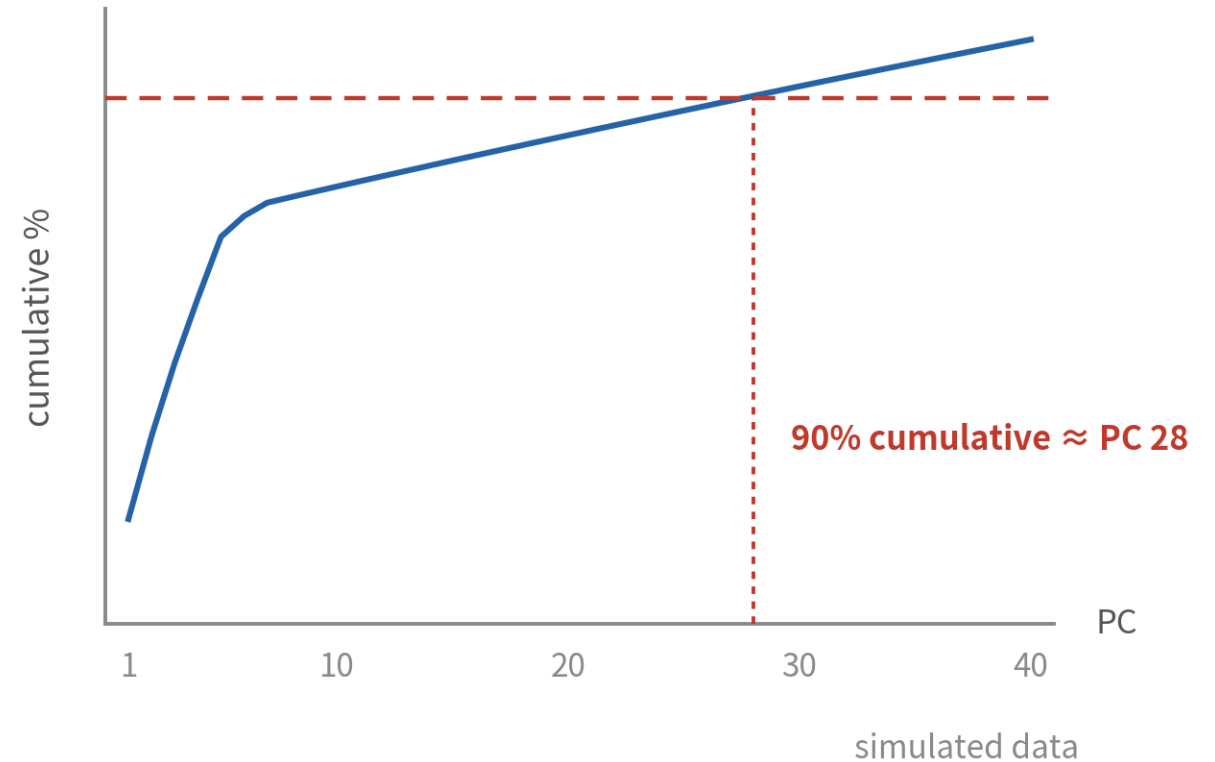
Cumulative 90% is a conservative ceiling; the <0.1% drop finds where the curve flattens — closer to the eyeballed elbow

The two cutoffs, drawn on the curves

variance explained per PC



cumulative view: a number you can write down



Numbers end reviewer arguments — but they assist the decision, they do not dictate it

FOR YOUR METHODS SECTION

**"Top 10 PCs were used (elbow at 7-12;
successive variance drop $<0.1\%$ at PC 13)."**

One sentence, two justifications — far sturdier than a bare 'we used 10 PCs'.

05

Method 3: the stability sweep

When unsure, run several nPCs and watch the downstream

Run the downstream at nPC = 5 through 30

```
for (d in c(5, 10, 15, 20, 30)) {  
  pbm c <- FindNeighbors(pbm c, dims = 1:d, verbose = FALSE)  
  pbm c <- FindClusters(pbm c, resolution = 0.5, verbose = FALSE)  
  pbm c@meta.data[[paste0("npc", d)]] <- Idents(pbm c)  
}  
sapply(pbm c@meta.data[paste0("npc", c(5, 10, 15, 20, 30))],  
       \(x) length(unique(x)))
```

```
npc5 npc10 npc15 npc20 npc30  
7    9    9    9    9
```

FindNeighbors / FindClusters star in B8 — today they are a black box; dims is the only knob

Did the members move? Cross-tabulate

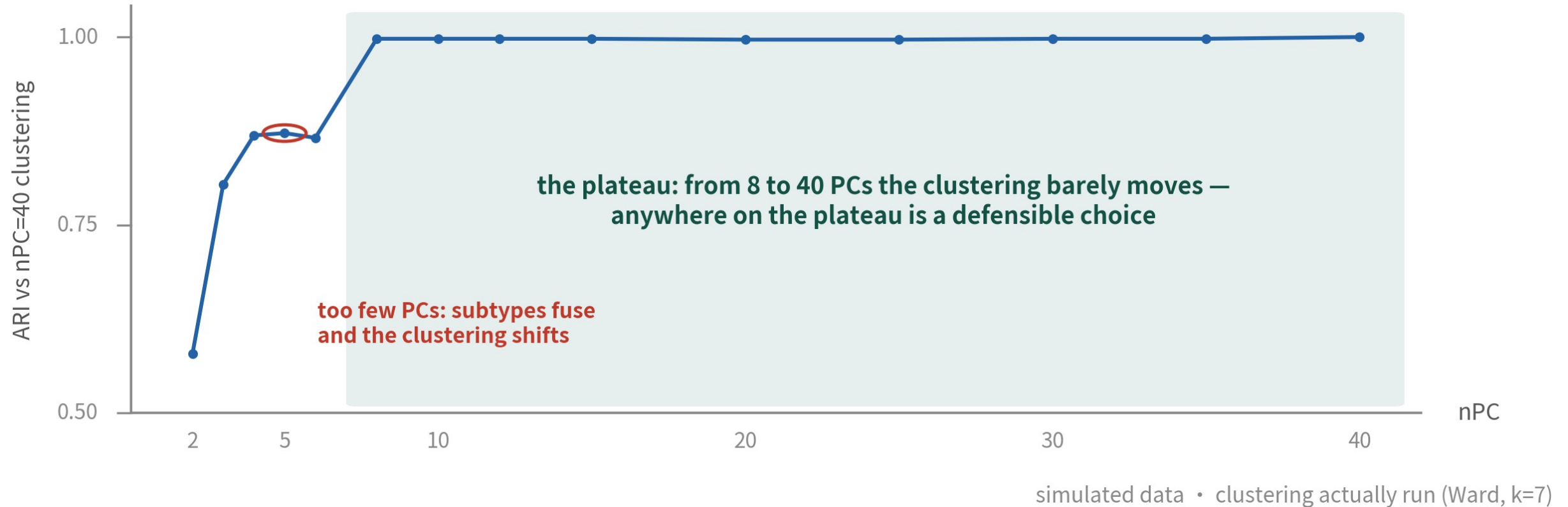
```
table(npc10 = pbm c$npc10, npc30 = pbm c$npc30)
```

```
      npc30  
npc10 0  1  2  3  4  
0 684  0  0  0  0  
1  0 481  0  0  0  
2  0  0 476  0  0  
3  0  0  0 344  0
```

(其餘列略：對角線之外幾乎全為 0)

A clean diagonal means nPC 10 and 30 agree. Always compare runs with a cross-tab, never by cluster ID

The stability curve: anywhere on the plateau works



Only below the plateau does clustering shift — the evidence that this choice is forgiving

Where did JackStraw go?

Every old tutorial had it; the v5 workflow dropped it

JackStraw (legacy)

Permute the data, test each PC

- statistically elegant: a p-value per PC
- slow; on big data nearly every PC turns 'significant'
- not supported after SCTransform

the v5 way

Elbow + heuristics + stability

- fast and sufficient: elbow range plus cutoffs
- forgiving: anywhere on the plateau is fine
- when unsure, sweep and watch the downstream

THE DECISION

PBMC 3k: all three methods point to 8-12.

We set nPC = 10 and write down why.

Complex tissues (tumour, development) elbow later — take 20-30. B8's dims = 1:10 inherits from here.

06

Where people get hurt

Two traps, both demonstrated for real

Trap 1: too few PCs

What it is

The first five PCs dominate the ElbowPlot, so `dims = 1:5` it is.



Why it happens

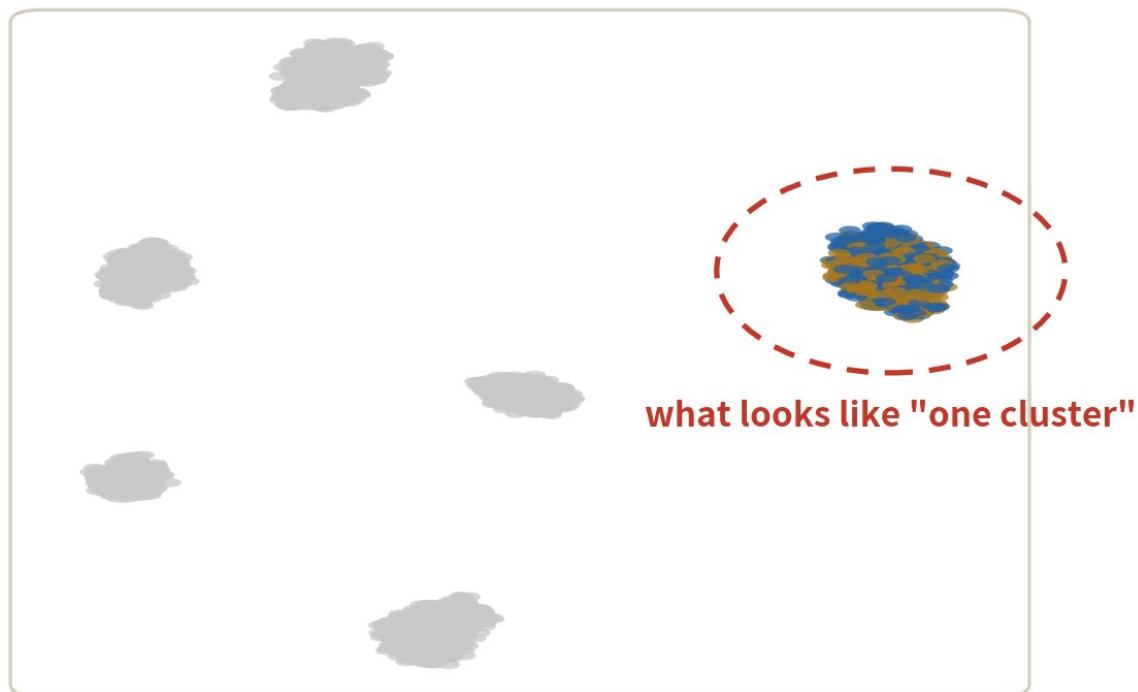
'Most variance is up front' sounds reasonable — and fewer dims run faster.

What it costs you

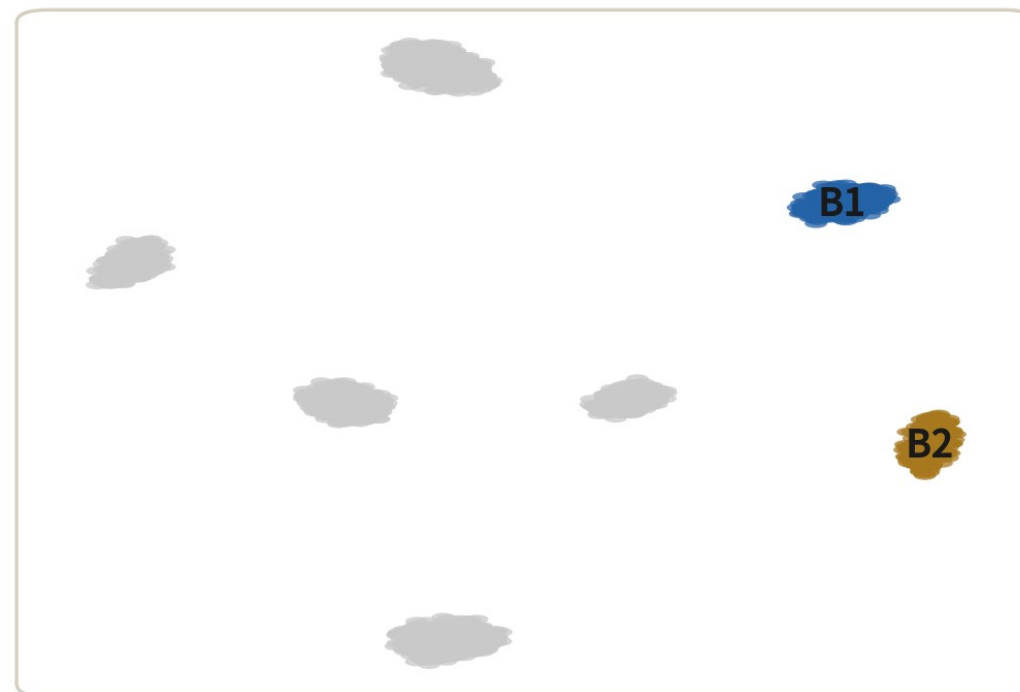
Subtype signal living in PCs 6-9 never enters the analysis: two subtypes fuse into one cluster, invisible to clustering, UMAP and annotation alike.

nPC = 5: the subtypes vanish in place

nPC = 5: subtypes B1 and B2 fuse



nPC = 20: the two subtypes separate



the subtype signal lives in PCs 6–9; with nPC=5 it never enters the analysis

grey = other cell types (simulated, actually run)

One-line check: if your nPC still sits on the steep slope of the ElbowPlot, do not stop there

Trap 2: reading PC1 as biology

What it is

PC1 orders the cells along a gradient, so the story begins: 'a differentiation trajectory!'



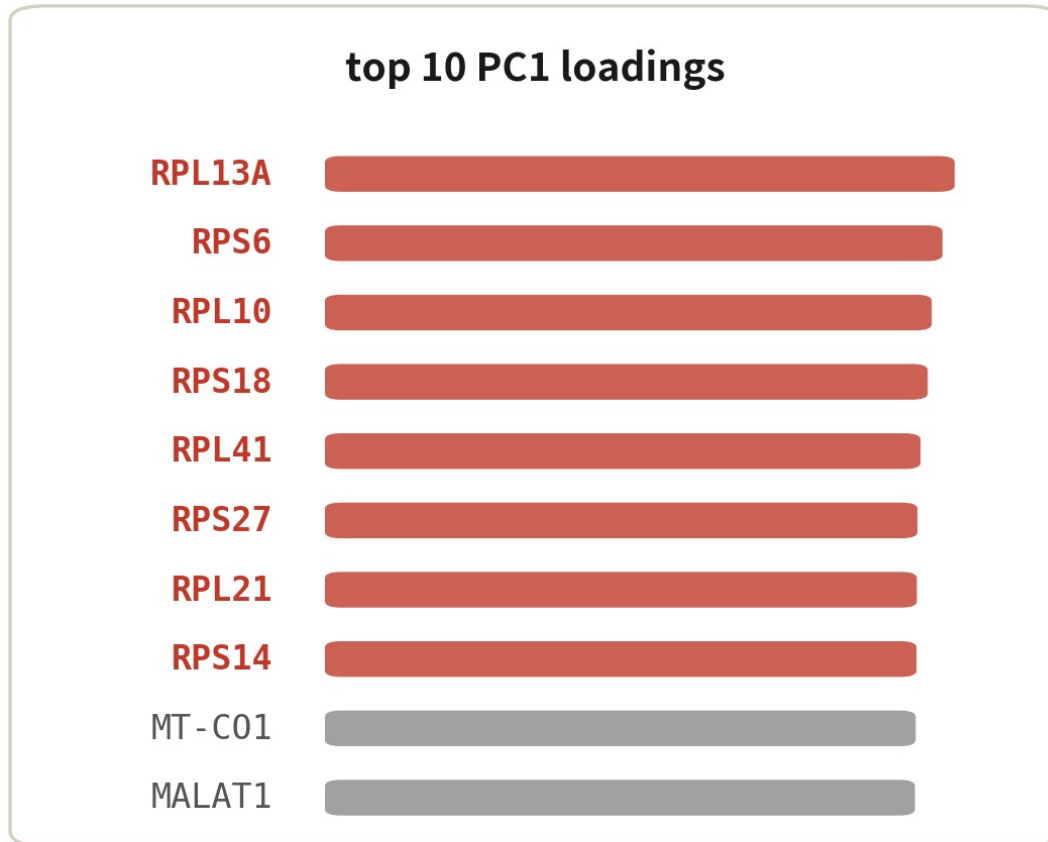
Why it happens

PCA faithfully captures the largest variance — nothing guarantees that variance is biology; technical variance qualifies too.

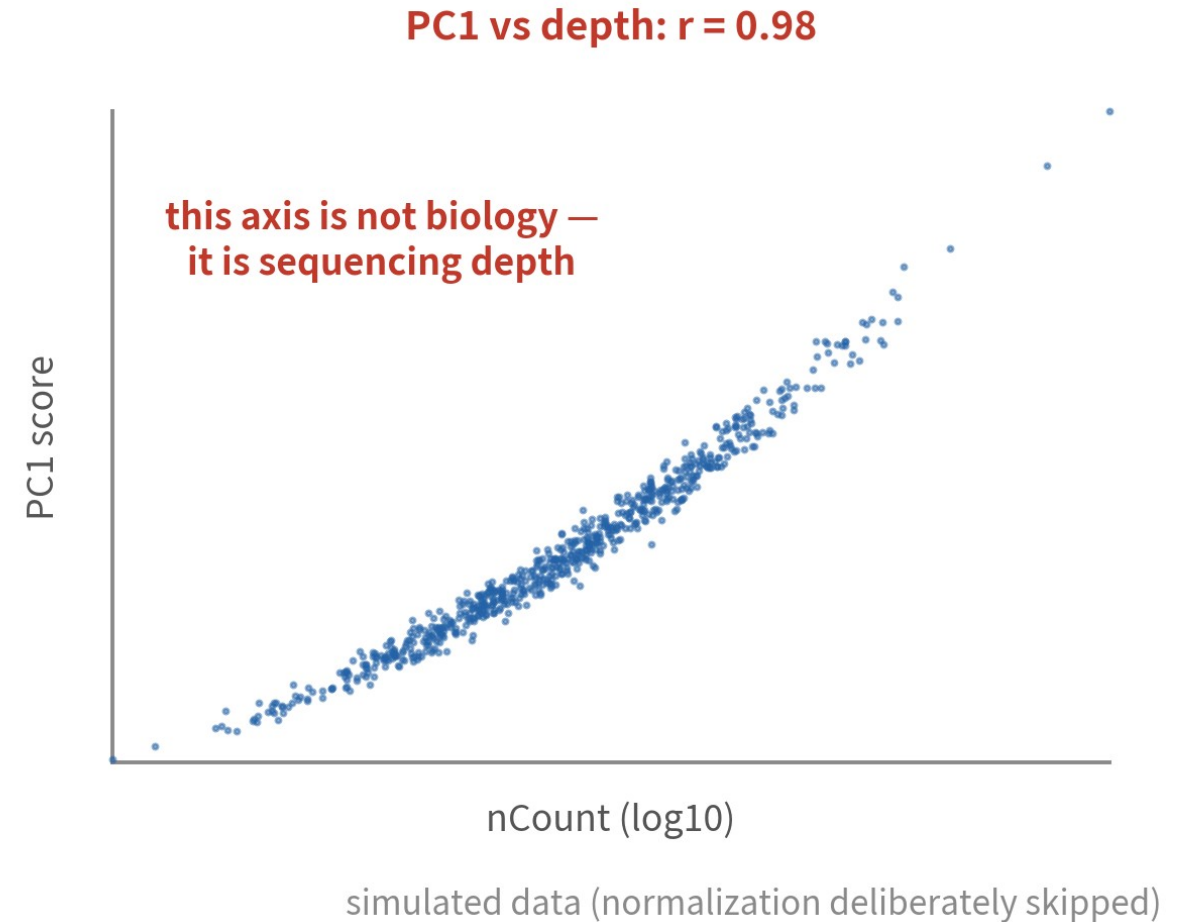
What it costs you

With normalization done poorly, PC1's loadings fill with ribosomal genes and correlate 0.9 with nCount — the 'trajectory' is sequencing depth.

A depth-hijacked PC1 looks like this



RP* = ribosomal protein genes



Top loadings all RPL/RPS, PC1 vs nCount at $r = 0.98$ — check first, interpret second

Three lines that block both traps

```
# 雷一: nPC 是否已在平原上 (不在就往多的取)
ElbowPlot(pbm c, ndims = 50)

# 雷二: PC1 是深度軸嗎? top loadings 被核糖體洗版嗎?
cor(pbm c$nCount_RNA, Embeddings(pbm c, "pca")[, 1])
mean(grep("^RP[LS]", top.pc1))
```

These live in R/B07_pca.R's main flow — run them on every dataset, not just after disasters

TRAPS, SUMMED UP

**Only one nPC mistake is truly fatal:
too few. Everything else is recoverable.**

Taking more costs a little noise and a few seconds; taking fewer can cost an invisible subpopulation.

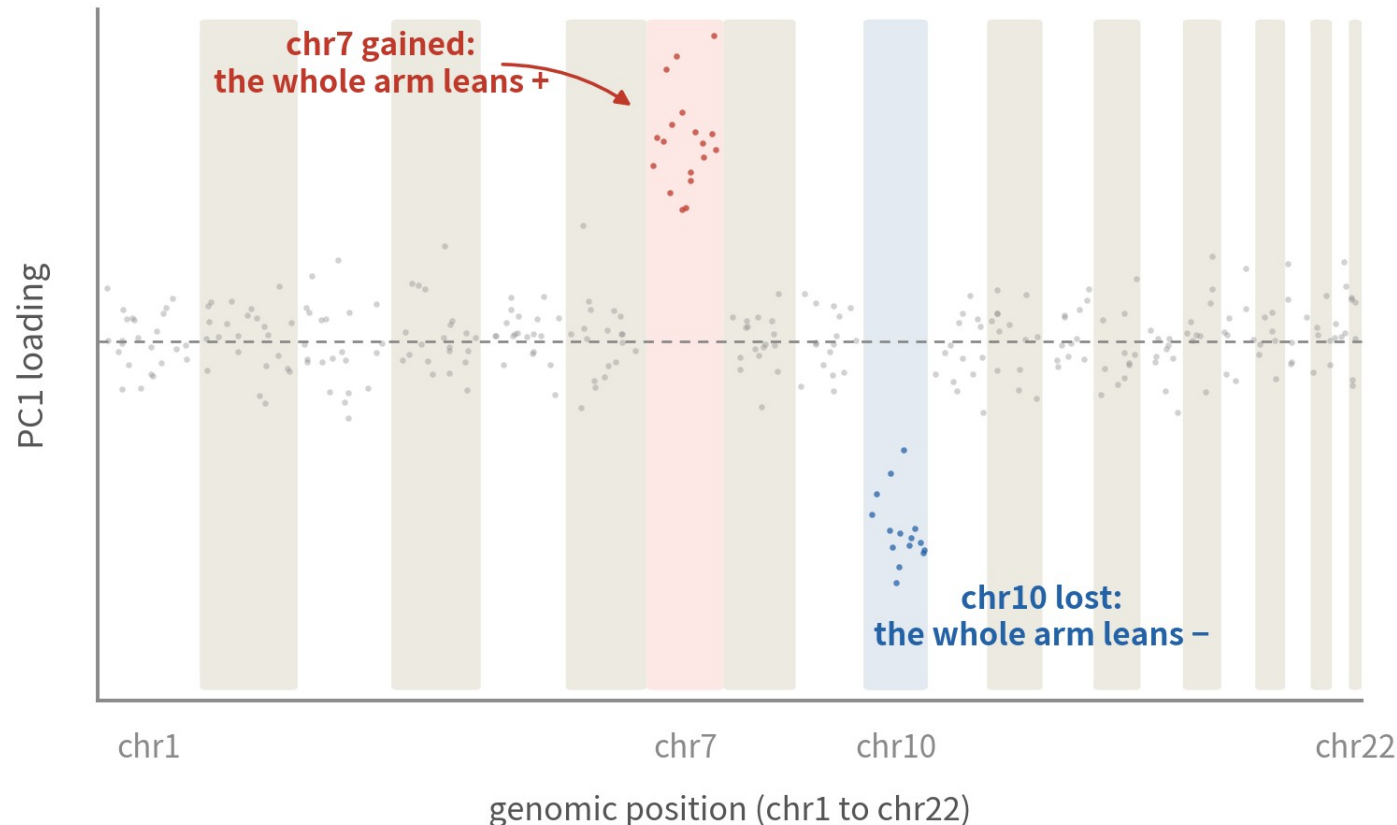
5+

Strategy Room: Complex Samples

The standard route is done — now imagine what you hold is not PBMC, but a tumour

CNV-driven PCs: chromosome events in the front row

Tumour PC1: loadings crowd onto chromosomes instead of scattering



illustrative data

Give loadings one more check: tag them with chromosomes. chr7 crowding the positive end, chr10 the negative — that axis is genome, not an expression program

the mechanism

malignant cells carry CNVs: gain an arm and every gene on it rises together — the very signal B16's inferCNV reads

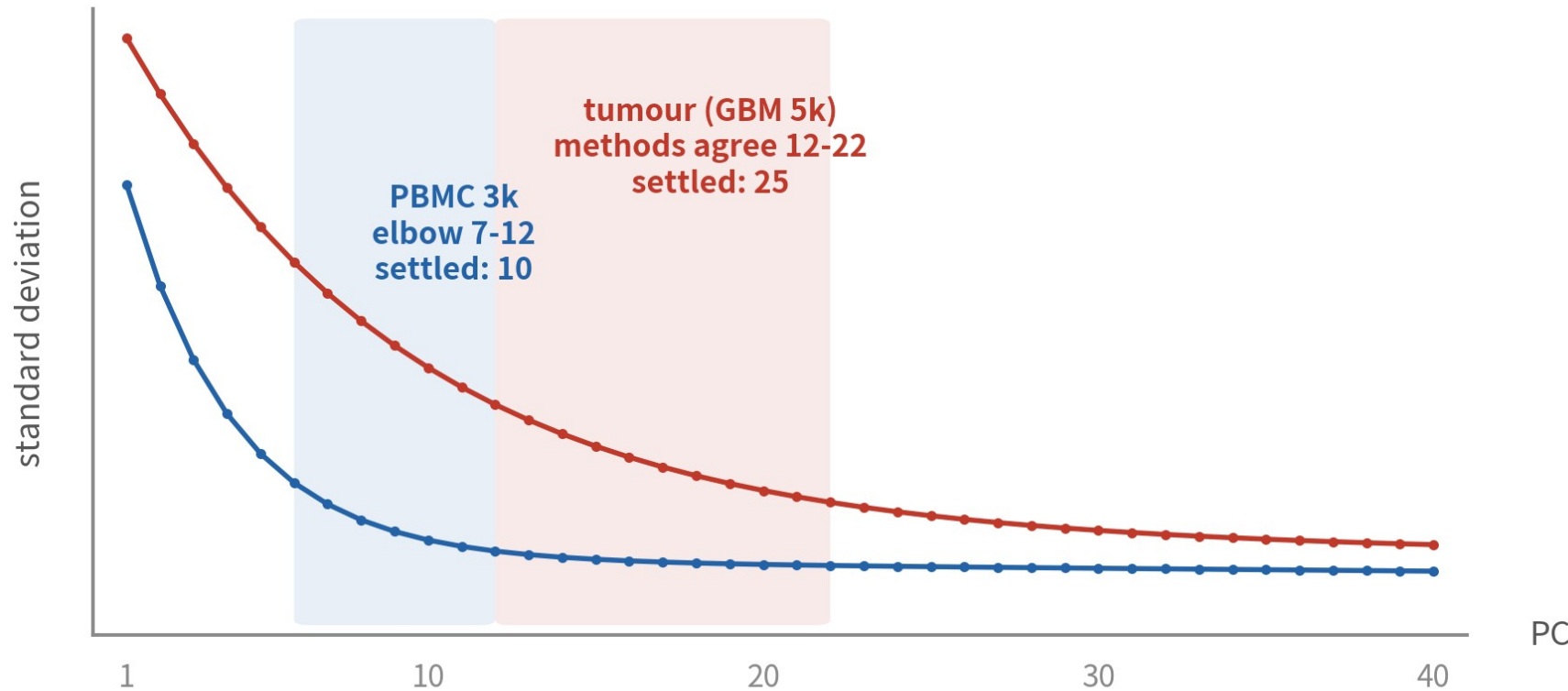
the check

tag top loadings with chromosomes:
scattered = expression program (PBMC's PC1)
crowded on one or two = a CNV axis

the reading

not a trap, don't discard — it is biology;
but note it down: one reason malignant cells
form per-patient islands (B11 and B16 return here)

nPC on tumours: a later elbow — take more



why later

more types, malignant states, cycling, patient effects — each claims axes

the decision

take dims to 20-30:
rare subtypes live in late PCs — too few = silent loss

walk the three methods again — a parameter is a hypothesis: number, reason, record

illustrative curves

Walk the same three methods again: on GBM 5k they intersect at 12-22, settled at 25 — the method carries over, the number is re-earned

Pooled PCA vs per-patient PCA

Two spaces, two jobs — the question decides which space you stand in

pooled PCA (all patients)

for structure shared across patients

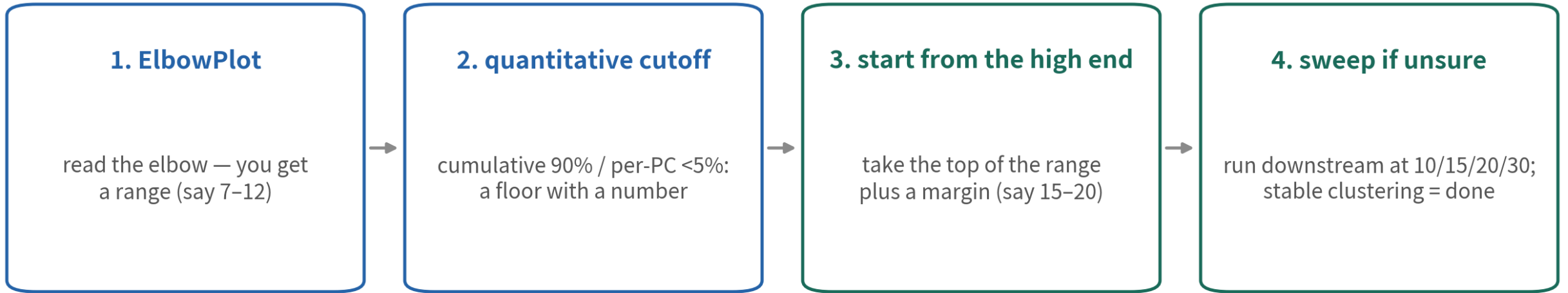
- immune and stromal types every patient shares: annotation and integration (B11) live here
- malignant cells sit in per-patient islands in this space — a phenomenon, not a bug
- the space for the atlas view and cross-patient alignment

per-patient PCA (one patient)

for structure inside one patient

- within a patient the CNV background is constant — patient effect gone, PCs go to states and subclusters
- re-pick HVGs, re-run PCA, re-choose nPC — the whole route, per patient
- the standard for fine-grained malignant analysis (hands-on in the Q series and B11)

The nPC decision flow



**when in doubt, take more: extra PCs add a little noise —
missing PCs can erase an entire subpopulation**

Four steps to a number with reasons — and when in doubt, take more

Episode checklist



RunPCA ran on the HVGs from B6's output



you printed the top loadings and can read them



you read the elbow and computed a quantitative cutoff



your nPC leans generous, with the reason written down



you checked PC1 for ribosomal / depth artefacts



downstream dims match the nPC chosen here

All six boxes ticked before you move to B8

ONE-LINE SUMMARY

**Three methods frame a range; settle on the generous side,
write down why — then stop worrying.**

On the plateau, 10 or 30 PCs hand you the same biology.

Check yourself 1

Why run PCA before clustering instead of computing distances on 20,000 genes?

- A 20,000 dimensions do not fit in memory
- B Per-gene noise cancels in the weighted sums, so distances reflect real type differences
- C PCA removes batch effects
- D Seurat requires RunPCA first

Answer B. Every gene carries sampling noise, and distances in 20,000 dimensions drown in it; a PC sums thousands of genes whose noise points in different directions and cancels. A is a side benefit, C is B11's job, D reverses cause and effect.

Check yourself 2

The elbow is fuzzy — anywhere from 7 to 12 looks plausible. The most reasonable next step?

- A Take 7 — fewer dimensions, cleaner data
- B Rerun RunPCA until the elbow sharpens
- C Take the high end plus margin (say 15), and sweep nPC if still unsure
- D Abandon PCA and cluster on all genes

Answer C. The elbow is inherently a range. Too few costs far more (lost subtypes) than too many (a little noise), so lean high; a sweep verifies the plateau directly. B does nothing — PCA is deterministic.

Check yourself 3

You are analysing tumour tissue — complex, with rare subtypes expected. How does your nPC strategy change?

- A Keep 10, same as PBMC
- B Take more (say 30-50) — subtype signal lives in later PCs
- C Take fewer, to avoid tumour noise
- D nPC does not affect rare subtype detection

Answer B. More complex populations need more expression programs; weak but crucial signals (rare subtypes, activation states) live in later PCs. Ten suits a small clean PBMC dataset — rerun the decision flow and the elbow will land later.

NEXT EPISODE

B8: Clustering — resolution sweeps and clustree

Distances are ready; next we turn them into clusters. The same data yields 6 clusters or 16 — the knob is called resolution, and everyone asks the same question: what value?